UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

UNIVERSITÉ SAINT JEAN — SAINT JEAN INGÉNIEUR

Année Académique 2025 – 2026

GUIDE COMPLET D'IMPLÉMENTATION

Conteneurisation et Gestion des Workloads avec OpenStack Magnum & Kubernetes

Rédigé par :
ESSONO EFFA Etienne Florian
&
NGNINTEDEM FEUPI Vaillant
Sous la supervision de M. NGUIMBUS Emmanuel

Installation : OpenStack via Kolla-Ansible | Ubuntu 24.04 LTS | VirtualBox

CHAPITRE 1 : CAHIER DES CHARGES

1.1 Contexte et Problématique

Ce projet vise à déployer une plateforme d'orchestration de conteneurs sur OpenStack en utilisant Magnum et Heat, sur Ubuntu 24.04 LTS dans un environnement VirtualBox. Suite aux difficultés rencontrées avec DevStack (blocage de init_placement, conflit OVN/q-agt, insuffisance de RAM), nous adoptons Kolla-Ansible — la méthode de déploiement recommandée en production par la communauté OpenStack, stable sur Ubuntu 24.04.

1.2 Besoins Fonctionnels

ID	Besoin	Description	Priorité
BF-01	Cluster Kubernetes via Magnum	Déployer un cluster K8s en un clic via openstack coe cluster create, avec template personnalisable	CRITIQUE
BF-02	Stack Hybride VM + Conteneurs	Déployer via Heat une VM (base de données) et un cluster K8s dans une même stack YAML	HAUTE
BF-03	Scaling du Cluster	Ajouter/supprimer des nœuds workers dynamiquement via openstack coe cluster resize	HAUTE
BF-04	Infrastructure as Code	Définir toute l'infrastructure en templates YAML HOT versionnables et reproductibles	HAUTE
BF-05	Gestion des Quotas	Gérer quotas CPU/RAM/Stockage par projet et nettoyer les ressources inutilisées	MOYENNE
BF-06	Accès Kubeconfig	Récupérer automatiquement le fichier kubeconfig pour accéder au cluster K8s	HAUTE
BF-07	Authentification RBAC	Sécuriser toutes les opérations via Keystone avec contrôle d'accès par rôles	CRITIQUE
BF-08	Rollback automatique	Déclencher un rollback si un déploiement Heat échoue partiellement	HAUTE

1.3 Besoins Non Fonctionnels

ID	Catégorie	Description
BNF-01	Performance	Création d'un cluster K8s de 3 nœuds en moins de 25 minutes
BNF-02	Stabilité	OpenStack via Kolla-Ansible est stable sur Ubuntu 24.04 (contrairement à DevStack)
BNF-03	Sécurité	Communications TLS entre services, certificats K8s auto-générés par Magnum

ID	Catégorie	Description
BNF-04	Scalabilité	Support jusqu'à 50 nœuds workers sans modification architecturale
BNF-05	Maintenabilité	Infrastructure entièrement définie en IaC (YAML) et versionnée via Git
BNF-06	Reproductibilité	Environnement re-déployable depuis zéro en moins de 3h via Kolla-Ansible
BNF-07	Observabilité	Logs accessibles via journalctl et openstack stack event list
BNF-08	Compatibilité	Images Fedora CoreOS pour les nœuds K8s, Ubuntu 24.04 pour le contrôleur

1.4 Estimation des Coûts

Estimation pour une entreprise moyenne (~50 développeurs) — environnement on-premise :

Ressource	Spécification	Coût estimé/mois	Remarque
Serveur Contrôleur	32 Go RAM, 16 vCPUs, 500 Go SSD	200 000 – 350 000 FCFA	Nœud contrôle OpenStack
Nœuds Calcul (×3)	32 Go RAM, 8 vCPUs, 300 Go SSD	250 000 – 400 000 FCFA/nœud	KVM pour les VMs K8s
Stockage (Cinder)	2 To NVMe	100 000 – 160 000 FCFA	Volumes persistants
Réseau	1 Gbps, switch manageable	50 000 – 80 000 FCFA	VLAN pour tenant networks
OS Ubuntu LTS	Par nœud	0 FCFA	Gratuit
Maintenance	1 ingénieur Cloud	350 000 – 500 000 FCFA	Compétences Kolla+K8s
TOTAL / MOIS		1 200 000 – 2 000 000 FCFA	≈ 1 850 – 3 050 €/mois

Option académique (config utilisée dans ce guide)

1 VM VirtualBox : 16 Go RAM, 8 vCPUs, 100 Go disque — coût = 0 FCFA
Swap de 8 Go activé obligatoirement (corrige les blocages rencontrés)
Mode All-in-One Kolla-Ansible : tous les services sur une seule machine

CHAPITRE 2 : ARCHITECTURE DU SYSTÈME

2.1 Architecture Logique — Services OpenStack

Service	Rôle	Interaction avec Magnum
Keystone	Identité et authentification RBAC	Valide chaque token avant toute opération
Glance	Stockage des images OS	Fournit l'image Fedora CoreOS aux nœuds K8s
Nova	Hyperviseur KVM — Calcul	Crée les VMs masters et workers du cluster
Neutron + OVN	Réseau SDN	Crée les réseaux tenant et les ports des nœuds
Cinder	Stockage bloc	Volumes persistants attachés aux nœuds K8s
Heat	Orchestration (templates HOT)	Reçoit les templates de Magnum, crée les ressources
Magnum	Gestionnaire de clusters COE	Génère les templates HOT et pilote Heat
Placement	Gestion des ressources	Alloue CPU/RAM aux instances Nova
Horizon	Dashboard Web	Interface graphique pour tout gérer
RabbitMQ	Bus de messages	Communication asynchrone entre services
MariaDB	Base de données	Stocke l'état de tous les services

2.2 Architecture Réseau du Projet

Réseau	CIDR	Rôle
Management	10.0.0.0/24	Communication interne entre services OpenStack
Tunnel (VXLAN/Geneve)	10.0.1.0/24	Trafic réseau des VMs (OVN)
External/Public	192.168.100.0/24	Floating IPs, accès depuis l'extérieur
Tenant (K8s)	10.11.12.0/24	Réseau privé du cluster Kubernetes

2.3 Flux de Création d'un Cluster Kubernetes

Étape	Service	Action
1	CLI → Keystone	Authentification, obtention d'un token
2	CLI → Magnum API	Envoi de la requête (nom, template, node_count)

Étape	Service	Action
3	Magnum API	Validation des quotas et du ClusterTemplate
4	Magnum → RabbitMQ	Message asynchrone au Magnum Conductor
5	Magnum Conductor	Génération dynamique du template Heat HOT
6	Magnum → Heat API	Soumission du template YAML à Heat
7	Heat Engine	Analyse du graphe de dépendances, création ordonnée
8	Heat → Nova	Création des VMs masters/workers via KVM
9	Heat → Neutron	Réseau tenant, ports, groupes de sécurité
10	cloud-init (VM)	Script d'auto-config : installation de Kubernetes
11	Magnum Conductor	Polling jusqu'à CREATE_COMPLETE
12	Magnum	Génération et stockage du kubeconfig
13	Utilisateur	Récupération via : openstack coe cluster config

CHAPITRE 3 : PRÉPARATION DE L'ENVIRONNEMENT UBUNTU 24.04

Pourquoi Kolla-Ansible et non DevStack ?

DevStack sur Ubuntu 24.04 génère des erreurs critiques rencontrées dans ce projet :

- Blocage de init_placement après 4h (manque de RAM sans swap)
- Conflit OVN vs q-agt : 'The q-agt/neutron-agt service must be disabled with OVN'
- Incompatibilités Python 3.12 avec certains composants OpenStack

Kolla-Ansible déploie OpenStack dans des conteneurs Docker : isolation totale, compatible Ubuntu 24.04, stable en production, et recommandé par la communauté OpenStack.

3.1 Prérequis Matériels (VM VirtualBox)

Composant	Minimum	Recommandé pour ce projet
RAM	12 Go	16 Go (+ swap 8 Go = 24 Go effectif)
CPU	4 vCPUs avec VT-x/AMD-V	8 vCPUs
Disque	80 Go	120 Go SSD
Réseau	1 interface NAT + 1 interface Host-Only	2 interfaces réseau
OS	Ubuntu 24.04 LTS	Ubuntu 24.04.4 LTS (version utilisée)

3.2 Configuration de VirtualBox AVANT de démarrer

Paramètres VirtualBox obligatoires (à faire VM éteinte)

1. Activer la virtualisation imbriquée : Paramètres → Système → Processeur → cocher 'Activer VT-x/AMD-V imbriqué'
2. RAM : allouer au moins 12 Go (12288 Mo) à la VM
3. CPU : allouer 4 vCPUs minimum
4. Réseau Carte 1 : NAT (pour accès internet pendant l'installation)
5. Réseau Carte 2 : Réseau Hôte uniquement / Host-Only (pour accès aux services depuis l'hôte)

Activer la virtualisation imbriquée via la ligne de commande

Sur le PC hôte (Windows PowerShell en administrateur, ou terminal Linux) :

```
# Sur Windows (PowerShell Administrateur) :
VBoxManage modifyvm "Ubuntu-OpenStack" --nested-hw-virt on
```

```
# Sur Linux hôte :
VBoxManage modifyvm "Ubuntu-OpenStack" --nested-hw-virt on

# Vérifier après démarrage de la VM :
egrep -c '(vmx|svm)' /proc/cpuinfo
# Résultat > 0 = virtualisation imbriquée activée
```

3.3 ÉTAPE 1 — Mise à jour et outils de base

```
# Se connecter en tant qu'utilisateur normal (ex: vboxuser)
# Mettre à jour le système
sudo apt update && sudo apt upgrade -y

# Installer les utilitaires essentiels
sudo apt install -y git curl wget vim net-tools python3-pip \
  python3-venv python3-dev build-essential \
  apt-transport-https ca-certificates gnupg lsb-release

# Vérifier la version Python (doit être 3.10+ )
python3 --version
# Résultat : Python 3.12.x (Ubuntu 24.04)
```

3.4 ÉTAPE 2 — Créer le Swap de 8 Go (OBLIGATOIRE)

Erreur corrigée : blocage de init_placement

Sans swap, les processus sont tués silencieusement par le kernel (OOM killer).
C'est la cause exacte du blocage observé pendant 4h sur init_placement.
Le swap de 8 Go résout définitivement ce problème.

```
# Vérifier qu'il n'y a pas de swap existant
free -h
# Si Échange = 0B → continuer

# Créer un fichier swap de 8 Go
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Rendre le swap permanent (survit aux redémarrages)
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

```
# Optimiser l'utilisation mémoire
sudo sysctl vm.swappiness=60
echo 'vm.swappiness=60' | sudo tee -a /etc/sysctl.conf

# Vérifier
free -h
# Résultat attendu : Échange: 8,0Gi
```

3.5 ÉTAPE 3 — Installer Docker (requis par Kolla-Ansible)

```
# Supprimer les anciennes versions de Docker si présentes
sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null

# Ajouter le dépôt officiel Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# Ajouter l'utilisateur courant au groupe docker
sudo usermod -aG docker $USER

# Activer Docker au démarrage
sudo systemctl enable --now docker

# Vérifier (déconnecter/reconnecter si 'permission denied')
docker --version
docker run hello-world
```

3.6 ÉTAPE 4 — Configurer le réseau

```
# Identifier les interfaces réseau
ip a
# Interface 1 (NAT)   : enp0s3 ou eth0  (accès internet)
# Interface 2 (H-Only): enp0s8 ou eth1  (accès depuis l'hôte)
```

```
# Récupérer l'IP de l'interface principale
hostname -I | awk '{print $1}'
# Noter cette IP — elle sera utilisée dans la config Kolla

# Désactiver le firewall UFW (Kolla gère ses propres règles)
sudo ufw disable
sudo ufw status
# Résultat attendu : Status: inactive

# Vérifier la résolution DNS
ping -c 3 8.8.8.8
ping -c 3 google.com
```

CHAPITRE 4 : INSTALLATION D'OPENSTACK VIA KOLLA-ANSIBLE

Présentation de Kolla-Ansible

Kolla-Ansible est l'outil officiel de la communauté OpenStack pour déployer tous les services dans des conteneurs Docker. Avantages par rapport à DevStack :

- Compatible Ubuntu 24.04 (testé et validé)
- Pas de conflit Python (chaque service est isolé dans son conteneur)
- Pas de blocage init_placement (ressources mieux gérées)
- Pas de conflit OVN/q-agt (configuration claire et séparée)
- Déploiement en production possible avec la même base

4.1 ÉTAPE 5 — Installer Kolla-Ansible

```
# Créer un environnement virtuel Python pour Kolla
python3 -m venv /opt/kolla-venv
source /opt/kolla-venv/bin/activate

# Mettre à jour pip
pip install -U pip

# Installer Kolla-Ansible (version Bobcat = OpenStack 2023.2, compatible Ubuntu
24.04)
pip install 'ansible-core>=2.15,<2.16'
pip install kolla-ansible

# Vérifier les installations
kolla-ansible --version
ansible --version

# Persister l'activation du venv dans .bashrc
echo 'source /opt/kolla-venv/bin/activate' >> ~/.bashrc
```

4.2 ÉTAPE 6 — Configurer Kolla-Ansible

Créer les répertoires et copier les fichiers de configuration

```
source /opt/kolla-venv/bin/activate

# Créer le répertoire de configuration Kolla
sudo mkdir -p /etc/kolla
sudo chown $USER:$USER /etc/kolla
```

```
# Copier les fichiers de configuration par défaut
cp -r /opt/kolla-venv/share/kolla-ansible/etc_examples/kolla/* /etc/kolla/

# Copier les fichiers d'inventaire Ansible
cp /opt/kolla-venv/share/kolla-ansible/ansible/inventory/all-in-one ~/all-in-one

# Vérifier la structure
ls /etc/kolla/
# Attendu : globals.yml  passwords.yml
```

Générer les mots de passe de tous les services

```
# Générer automatiquement tous les mots de passe OpenStack
kolla-genpwd

# Vérifier (chaque service a maintenant un mot de passe unique)
grep -E 'keystone|database|rabbit' /etc/kolla/passwords.yml | head -10
```

Configurer globals.yml — Fichier principal de configuration

```
# Récupérer ton IP principale
MY_IP=$(hostname -I | awk '{print $1}')
echo $MY_IP  # Noter cette valeur !

# Identifier l'interface réseau principale
ip route | grep default | awk '{print $5}'
# Exemple de résultat : enp0s3 (utiliser ce nom dans la config)
```

Éditer /etc/kolla/globals.yml :

```
sudo nano /etc/kolla/globals.yml

# Effacer le contenu existant et coller la configuration suivante :
# (Remplacer enp0s3 par ton interface et 192.168.X.X par ton IP)
# ============================================================
# /etc/kolla/globals.yml — Configuration Kolla-Ansible
# ============================================================

# Image Docker de base
kolla_base_distro: 'ubuntu'
kolla_install_type: 'source'
openstack_release: '2023.2'

# Réseau — ADAPTER À TON ENVIRONNEMENT
# network_interface : interface connectée au réseau interne
network_interface: 'enp0s3'
```

```
# neutron_external_interface : interface pour les Floating IPs
# (peut être la même que network_interface en mode All-in-One)
neutron_external_interface: 'enp0s8'

# IP virtuelle du contrôleur OpenStack (= ton IP principale)
kolla_internal_vip_address: '192.168.X.X'

# Mode All-in-One (contrôle + calcul sur la même machine)
enable_haproxy: 'no'

# ============================================================
# SERVICES À ACTIVER
# ============================================================

# Services de base (toujours actifs)
enable_keystone: 'yes'
enable_glance: 'yes'
enable_nova: 'yes'
enable_placement: 'yes'

# Neutron avec OVN (corrige le conflit q-agt rencontré avec DevStack)
enable_neutron: 'yes'
neutron_plugin_agent: 'ovn'
enable_neutron_provider_networks: 'yes'

# Heat (Orchestration — OBLIGATOIRE pour Magnum)
enable_heat: 'yes'

# Magnum (Gestion des clusters Kubernetes)
enable_magnum: 'yes'

# Horizon (Interface Web)
enable_horizon: 'yes'

# Cinder (Stockage bloc pour les volumes K8s)
enable_cinder: 'yes'
enable_cinder_backend_lvm: 'yes'
cinder_volume_group: 'cinder-volumes'

# ============================================================
# HYPERVISEUR
# ============================================================
nova_compute_virt_type: 'kvm'
```

```
# ============================================================
# DOCKER REGISTRY
# ============================================================
docker_registry: 'quay.io'
docker_registry_insecure: 'no'
```

4.3 ÉTAPE 7 — Préparer le volume Cinder (LVM)

```
# Vérifier l'espace disponible
df -h
lsblk

# Créer un fichier loopback de 30 Go pour Cinder (si pas de disque secondaire)
sudo dd if=/dev/zero of=/var/lib/cinder-volumes bs=1G count=30 status=progress

# Créer un device loopback
sudo losetup /dev/loop0 /var/lib/cinder-volumes

# Créer le Physical Volume LVM
sudo pvcreate /dev/loop0

# Créer le Volume Group 'cinder-volumes'
sudo vgcreate cinder-volumes /dev/loop0

# Vérifier
sudo vgdisplay cinder-volumes
# Résultat : VG Name = cinder-volumes, VG Status = resizable

# Rendre le loopback permanent au démarrage
cat << 'EOF' | sudo tee /etc/rc.local
#!/bin/bash
losetup /dev/loop0 /var/lib/cinder-volumes
vgchange -ay cinder-volumes
exit 0
EOF
sudo chmod +x /etc/rc.local
```

4.4 ÉTAPE 8 — Préparer les nœuds avec Kolla-Ansible

```
source /opt/kolla-venv/bin/activate

# Installer les dépendances Ansible pour OpenStack
kolla-ansible install-deps
```

```
# Vérifier la configuration (dry-run)
ansible -i ~/all-in-one all -m ping
# Résultat attendu : localhost | SUCCESS

# Préparer les nœuds (installe Docker, configure le réseau, etc.)
kolla-ansible -i ~/all-in-one bootstrap-servers

# Vérifier les prérequis
kolla-ansible -i ~/all-in-one prechecks
# Tous les checks doivent passer AVANT de continuer
```

4.5 ÉTAPE 9 — Déployer OpenStack

Durée estimée du déploiement

Le déploiement complet prend entre 30 et 60 minutes selon la vitesse internet et les ressources de la machine. Ne pas interrompre le processus. Surveiller la progression dans un second terminal.

```
source /opt/kolla-venv/bin/activate

# LANCER LE DÉPLOIEMENT COMPLET
kolla-ansible -i ~/all-in-one deploy

# ---------------------------------------------------------
# Dans un SECOND terminal — Surveiller la progression :
# ---------------------------------------------------------
watch -n 30 'docker ps --format "table {{.Names}}\t{{.Status}}" | head -30'

# Surveiller les ressources (éviter le blocage RAM)
watch -n 15 'free -h && echo --- && df -h /'

# ---------------------------------------------------------
# RÉSULTAT ATTENDU EN FIN DE DÉPLOIEMENT :
# ---------------------------------------------------------
# PLAY RECAP *****
# localhost : ok=XX  changed=XX  unreachable=0  failed=0
#
# Les failed=0 confirme que tout s'est bien passé
```

4.6 ÉTAPE 10 — Post-déploiement et vérification

```
source /opt/kolla-venv/bin/activate

# Générer le fichier openrc (variables d'environnement OpenStack)
```

```
kolla-ansible -i ~/all-in-one post-deploy

# Charger les variables d'environnement admin
source /etc/kolla/admin-openrc.sh

# Vérifier que tous les services sont actifs
openstack service list

# Vérifier les endpoints
openstack endpoint list | grep -E 'magnum|heat|nova|neutron'

# Vérifier Nova
openstack compute service list

# Vérifier Neutron + OVN
openstack network agent list

# Vérifier Magnum
openstack coe service list
# Attendu : magnum-api et magnum-conductor en état 'up'

# Accéder au Dashboard Horizon
echo 'Horizon disponible sur : http://'$(hostname -I | awk '{print $1}')'/horizon'
echo 'Login : admin'
echo 'Mot de passe :' $(grep keystone_admin_password /etc/kolla/passwords.yml |
awk '{print $2}')
```

CHAPITRE 5 : CONFIGURATION INITIALE D'OPENSTACK

5.1 ÉTAPE 11 — Initialiser le réseau externe

```
source /etc/kolla/admin-openrc.sh

# Créer le réseau provider (externe / floating IPs)
openstack network create --share --external \
  --provider-physical-network physnet1 \
  --provider-network-type flat \
  public

# Créer le sous-réseau public (adapter selon ton réseau)
openstack subnet create public-subnet \
  --network public \
  --subnet-range 192.168.100.0/24 \
  --gateway 192.168.100.1 \
  --dns-nameserver 8.8.8.8 \
  --allocation-pool start=192.168.100.100,end=192.168.100.200 \
  --ip-version 4 \
  --no-dhcp

# Vérifier
openstack network list
openstack subnet list
```

5.2 ÉTAPE 12 — Créer un projet et un utilisateur dédié

```
# Créer un projet pour le projet K8s
openstack project create --description 'Projet Kubernetes USJ' k8s-project

# Créer un utilisateur
openstack user create k8s-admin \
  --password K8sAdmin123! \
  --project k8s-project

# Attribuer le rôle admin au projet
openstack role add --project k8s-project --user k8s-admin admin
openstack role add --project k8s-project --user k8s-admin _member_

# Créer le fichier openrc pour cet utilisateur
cat > ~/k8s-openrc.sh << 'EOF'
export OS_PROJECT_DOMAIN_NAME=Default
```

```
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_NAME=k8s-project
export OS_USERNAME=k8s-admin
export OS_PASSWORD=K8sAdmin123!
export OS_AUTH_URL=http://192.168.X.X:5000/v3
export OS_IDENTITY_API_VERSION=3
export OS_IMAGE_API_VERSION=2
EOF

# Définir des quotas suffisants pour les clusters K8s
openstack quota set \
  --cores 32 \
  --ram 65536 \
  --instances 20 \
  --volumes 30 \
  --gigabytes 500 \
  --floating-ips 15 \
  k8s-project
```

5.3 ÉTAPE 13 — Créer les gabarits Nova (flavors)

```
source /etc/kolla/admin-openrc.sh

# Gabarit pour les nœuds workers K8s (2 vCPU, 4 Go RAM, 40 Go disque)
openstack flavor create k8s.worker \
  --vcpus 2 --ram 4096 --disk 40 --public

# Gabarit pour les nœuds masters K8s (2 vCPU, 4 Go RAM, 40 Go disque)
openstack flavor create k8s.master \
  --vcpus 2 --ram 4096 --disk 40 --public

# Gabarit pour les VMs classiques (base de données, etc.)
openstack flavor create vm.medium \
  --vcpus 2 --ram 4096 --disk 50 --public

# Vérifier
openstack flavor list
```

5.4 ÉTAPE 14 — Télécharger et enregistrer l'image Fedora CoreOS

Pourquoi Fedora CoreOS ?

Fedora CoreOS est l'image recommandée par Magnum pour les nœuds Kubernetes.
Elle est pré-configurée avec containerd et les agents cloud-init nécessaires

pour rejoindre automatiquement un cluster K8s lors du premier démarrage.
La propriété os_distro=fedora-coreos est OBLIGATOIRE pour que Magnum la reconnaisse.

```
# Créer un répertoire pour les images
mkdir -p ~/openstack-images && cd ~/openstack-images

# Télécharger Fedora CoreOS (image stable pour OpenStack)
wget -O fedora-coreos.qcow2.xz \
  'https://builds.coreos.fedoraproject.org/prod/streams/stable/builds/
38.20230918.3.0/x86_64/fedora-coreos-38.20230918.3.0-openstack.x86_64.qcow2.xz'

# Décompresser
unxz fedora-coreos.qcow2.xz
ls -lh *.qcow2
# Résultat attendu : fichier .qcow2 entre 700 Mo et 1.2 Go

# Uploader dans Glance avec la propriété os_distro obligatoire
source /etc/kolla/admin-openrc.sh

openstack image create fedora-coreos-latest \
  --file ~/openstack-images/fedora-coreos.qcow2 \
  --disk-format qcow2 \
  --container-format bare \
  --public \
  --property os_distro=fedora-coreos

# Vérifier : status doit être 'active'
openstack image show fedora-coreos-latest | grep -E 'status|os_distro|size'

# Télécharger aussi Ubuntu 22.04 pour les VMs classiques (stack hybride)
wget -O ubuntu-22.04.qcow2 \
  'https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img'

openstack image create ubuntu-22.04 \
  --file ~/openstack-images/ubuntu-22.04.qcow2 \
  --disk-format qcow2 --container-format bare --public
```

5.5 ÉTAPE 15 — Créer une paire de clés SSH

```
# Générer une paire de clés SSH dédiée au projet
ssh-keygen -t rsa -b 4096 -N '' -f ~/.ssh/openstack_k8s_key

# Enregistrer la clé publique dans OpenStack
source /etc/kolla/admin-openrc.sh
openstack keypair create \
```

```
  --public-key ~/.ssh/openstack_k8s_key.pub \
  k8s-keypair

# Vérifier
openstack keypair list
```

CHAPITRE 6 : MAGNUM — DÉPLOIEMENT DE CLUSTERS KUBERNETES

6.1 ÉTAPE 16 — Créer le ClusterTemplate Kubernetes

Qu'est-ce qu'un ClusterTemplate ?

Un ClusterTemplate est un modèle (moule) définissant les paramètres par défaut d'un cluster : image OS, réseau, driver réseau conteneur (CNI), gabarits Nova, etc. Un seul template peut servir à créer plusieurs clusters aux configurations identiques.

```
source /etc/kolla/admin-openrc.sh

# Créer le ClusterTemplate Kubernetes avec Calico (CNI recommandé)
openstack coe cluster template create k8s-template \
  --image fedora-coreos-latest \
  --keypair k8s-keypair \
  --external-network public \
  --flavor k8s.worker \
  --master-flavor k8s.master \
  --docker-volume-size 50 \
  --network-driver calico \
  --coe kubernetes \
  --volume-driver cinder \
  --labels kube_tag=v1.26.4,container_runtime=containerd \
  --dns-nameserver 8.8.8.8 \
  --public \
  --master-lb-enabled \
  --floating-ip-enabled

# Vérifier le template créé
openstack coe cluster template list
openstack coe cluster template show k8s-template
```

Paramètre	Valeur	Rôle
--image	fedora-coreos-latest	Image OS des nœuds (os_distro=fedora-coreos obligatoire)
--keypair	k8s-keypair	Clé SSH pour accéder aux nœuds si nécessaire
--external-network	public	Réseau pour les Floating IPs des masters
--flavor	k8s.worker	Gabarit Nova pour les nœuds workers
--master-flavor	k8s.master	Gabarit Nova pour les nœuds masters (control

Paramètre	Valeur	Rôle
		plane)
--docker-volume-size	50 Go	Volume Cinder pour le runtime de conteneurs
--network-driver	calico	Driver réseau CNI des conteneurs
--coe	kubernetes	Container Orchestration Engine (k8s ou swarm)
--labels	kube_tag=v1.26.4	Version de Kubernetes et runtime (containerd)
--master-lb-enabled	oui	Load balancer Octavia devant les masters
--floating-ip-enabled	oui	Attribution de Floating IPs aux nœuds

6.2 ÉTAPE 17 — Créer un Cluster Kubernetes

```
source /etc/kolla/admin-openrc.sh

# Créer un cluster K8s : 1 master + 3 workers
openstack coe cluster create mon-cluster-k8s \
  --cluster-template k8s-template \
  --master-count 1 \
  --node-count 3 \
  --timeout 60

# Résultat attendu :
# Request to create cluster mon-cluster-k8s accepted
# <uuid-du-cluster>

# Surveiller la création (toutes les 30 secondes)
watch -n 30 'openstack coe cluster list'

# Statuts successifs :
#   CREATE_IN_PROGRESS  →  (10 à 25 min)  →  CREATE_COMPLETE
#   En cas d'erreur     →  CREATE_FAILED

# Voir le détail de la progression via Heat
openstack stack list | grep -i cluster
openstack stack event list <NOM_STACK_HEAT> --nested-depth 2
```

6.3 ÉTAPE 18 — Accéder au cluster avec kubectl

```
# Installer kubectl (si non disponible)
curl -LO 'https://dl.k8s.io/release/v1.26.4/bin/linux/amd64/kubectl'
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

```
kubectl version --client

# Créer le répertoire kubeconfig
mkdir -p ~/.kube

# Récupérer le kubeconfig depuis Magnum
openstack coe cluster config mon-cluster-k8s \
  --dir ~/.kube \
  --force

# Définir la variable KUBECONFIG
export KUBECONFIG=~/.kube/config
echo 'export KUBECONFIG=~/.kube/config' >> ~/.bashrc

# Tester la connexion au cluster
kubectl cluster-info
kubectl get nodes
# Résultat attendu : 1 master + 3 workers en statut 'Ready'

kubectl get namespaces
kubectl get pods -A
# Vérifier que les pods système (coredns, calico, etc.) sont Running
```

6.4 ÉTAPE 19 — Déployer une application de test

```
# Déployer Nginx comme application de test
kubectl create deployment nginx-test --image=nginx:latest --replicas=3

# Exposer via un Service LoadBalancer
kubectl expose deployment nginx-test \
  --port=80 --target-port=80 \
  --type=LoadBalancer --name=nginx-service

# Surveiller le déploiement
kubectl rollout status deployment/nginx-test
kubectl get pods -l app=nginx-test
kubectl get service nginx-service

# Une fois que EXTERNAL-IP est attribuée :
NGINX_IP=$(kubectl get svc nginx-service -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')
curl http://$NGINX_IP
# Résultat attendu : page HTML de bienvenue Nginx

# Nettoyer le test
```

```
kubectl delete service nginx-service
kubectl delete deployment nginx-test
```

6.5 ÉTAPE 20 — Scaling d'un cluster

Scale-Up : Augmenter le nombre de nœuds

```
# Voir le nombre actuel de nœuds
openstack coe cluster show mon-cluster-k8s | grep node_count

# Passer de 3 à 5 nœuds (scale-up)
openstack coe cluster resize mon-cluster-k8s --node-count 5

# Surveiller
watch -n 30 'openstack coe cluster show mon-cluster-k8s | grep -E "status|
node_count"'

# Vérifier dans Kubernetes
kubectl get nodes
# Résultat attendu : 5 workers + 1 master en statut 'Ready'
```

Scale-Down : Réduire le nombre de nœuds

```
# Réduire à 2 nœuds (scale-down)
openstack coe cluster resize mon-cluster-k8s --node-count 2

# K8s drainera les pods des nœuds supprimés automatiquement
kubectl get nodes
# Résultat attendu : 2 workers + 1 master en statut 'Ready'
```

CHAPITRE 7 : HEAT — STACK HYBRIDE VM + CLUSTER KUBERNETES

Objectif du chapitre

Déployer en une seule commande une infrastructure hybride composée de :

- Une VM classique hébergeant une base de données PostgreSQL (tier données)
- Un cluster Kubernetes via Magnum pour les microservices (tier application)

Le tout défini dans un unique fichier YAML (template HOT) versionnable.

7.1 ÉTAPE 21 — Créer le template HOT hybride

```
mkdir -p ~/heat-templates
cat > ~/heat-templates/hybrid-stack.yaml << 'YAML_EOF'
heat_template_version: 2021-04-16

description: >
  Stack Hybride USJ : VM PostgreSQL + Cluster Kubernetes via Magnum.
  Déploie une infrastructure 3-tiers complète en une seule commande.

parameters:
  keypair_name:
    type: string
    description: Paire de clés SSH
    default: k8s-keypair

  db_flavor:
    type: string
    description: Gabarit Nova pour la VM base de données
    default: vm.medium

  db_image:
    type: string
    description: Image Ubuntu pour la VM de BDD
    default: ubuntu-22.04

  k8s_template_name:
    type: string
    description: Nom du ClusterTemplate Magnum
    default: k8s-template

  k8s_node_count:
    type: number
```

```
      description: Nombre de nœuds workers K8s
      default: 2

    external_network:
      type: string
      description: Réseau externe pour les Floating IPs
      default: public

  resources:

    # ========================
    # RÉSEAU PRIVÉ PARTAGÉ
    # ========================
    private_network:
      type: OS::Neutron::Net
      properties:
        name: hybrid-private-net

    private_subnet:
      type: OS::Neutron::Subnet
      properties:
        network: { get_resource: private_network }
        cidr: 10.20.30.0/24
        gateway_ip: 10.20.30.1
        dns_nameservers: [8.8.8.8, 8.8.4.4]

    router:
      type: OS::Neutron::Router
      properties:
        external_gateway_info:
          network: { get_param: external_network }

    router_interface:
      type: OS::Neutron::RouterInterface
      properties:
        router: { get_resource: router }
        subnet: { get_resource: private_subnet }

    # ========================
    # GROUPE DE SÉCURITÉ BDD
    # ========================
    db_security_group:
      type: OS::Neutron::SecurityGroup
      properties:
        name: db-security-group
        rules:
```

```
        - protocol: tcp
          port_range_min: 5432
          port_range_max: 5432
          remote_ip_prefix: 10.20.30.0/24
        - protocol: tcp
          port_range_min: 22
          port_range_max: 22
          remote_ip_prefix: 0.0.0.0/0
        - protocol: icmp
          remote_ip_prefix: 0.0.0.0/0

  # =========================
  # VM BASE DE DONNÉES
  # =========================
  db_port:
    type: OS::Neutron::Port
    properties:
      network: { get_resource: private_network }
      security_groups: [{ get_resource: db_security_group }]

  db_server:
    type: OS::Nova::Server
    properties:
      name: db-postgresql
      image: { get_param: db_image }
      flavor: { get_param: db_flavor }
      key_name: { get_param: keypair_name }
      networks:
        - port: { get_resource: db_port }
      user_data_format: RAW
      user_data: |
        #!/bin/bash
        apt-get update && apt-get install -y postgresql
        systemctl enable postgresql && systemctl start postgresql
        sudo -u postgres psql -c "CREATE DATABASE appdb;"
        sudo -u postgres psql -c "CREATE USER appuser WITH PASSWORD
'AppPass123!';"
        sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE appdb TO
appuser;"
        sed -i "s/#listen_addresses.*/listen_addresses = '*'/"
/etc/postgresql/14/main/postgresql.conf
        echo "host all all 10.20.30.0/24 md5" >>
/etc/postgresql/14/main/pg_hba.conf
        systemctl restart postgresql

  db_floating_ip:
    type: OS::Neutron::FloatingIP
```

```
    properties:
      floating_network: { get_param: external_network }
      port_id: { get_resource: db_port }

  # ========================
  # CLUSTER KUBERNETES
  # ========================
  k8s_cluster:
    type: OS::Magnum::Cluster
    depends_on: [router_interface]
    properties:
      name: hybrid-k8s-cluster
      cluster_template_id: { get_param: k8s_template_name }
      node_count: { get_param: k8s_node_count }
      master_count: 1
      timeout: 60

outputs:
  db_private_ip:
    description: IP privée du serveur PostgreSQL
    value: { get_attr: [db_server, first_address] }

  db_public_ip:
    description: Floating IP du serveur PostgreSQL
    value: { get_attr: [db_floating_ip, floating_ip_address] }

  k8s_cluster_id:
    description: UUID du cluster Kubernetes
    value: { get_resource: k8s_cluster }

  connection_string:
    description: Chaîne de connexion PostgreSQL
    value:
      str_replace:
        template: postgresql://appuser:AppPass123!@DB_IP/appdb
        params:
          DB_IP: { get_attr: [db_server, first_address] }
YAML_EOF

echo 'Template HOT créé : ~/heat-templates/hybrid-stack.yaml'
```

7.2 ÉTAPE 22 — Déployer la Stack Hybride

```
source /etc/kolla/admin-openrc.sh
```

```
# Valider le template AVANT de déployer (évite les erreurs)
openstack orchestration template validate \
  -t ~/heat-templates/hybrid-stack.yaml
# Résultat attendu : le YAML est affiché sans erreur

# Déployer la stack hybride
openstack stack create hybrid-infra \
  -t ~/heat-templates/hybrid-stack.yaml \
  --parameter keypair_name=k8s-keypair \
  --parameter k8s_node_count=2 \
  --rollback-on-failure

# --rollback-on-failure : rollback auto si une ressource échoue

# Surveiller le déploiement
watch -n 30 'openstack stack show hybrid-infra | grep -E "stack_status"'

# Voir les événements en temps réel
openstack stack event list hybrid-infra --follow

# Statuts attendus :
#   CREATE_IN_PROGRESS → (15-30 min) → CREATE_COMPLETE
```

7.3 ÉTAPE 23 — Récupérer les sorties et utiliser la stack

```
# Afficher toutes les sorties
openstack stack output list hybrid-infra

# Récupérer l'IP du serveur PostgreSQL
DB_IP=$(openstack stack output show hybrid-infra db_public_ip \
  -f value -c output_value)
echo 'IP du serveur BDD : '$DB_IP

# Récupérer l'ID du cluster K8s
K8S_ID=$(openstack stack output show hybrid-infra k8s_cluster_id \
  -f value -c output_value)
echo 'ID Cluster K8s : '$K8S_ID

# Afficher la chaîne de connexion PostgreSQL
openstack stack output show hybrid-infra connection_string

# Se connecter au serveur PostgreSQL via SSH
ssh -i ~/.ssh/openstack_k8s_key ubuntu@$DB_IP
```

```
# Tester la connexion PostgreSQL depuis le cluster K8s
# (depuis un pod Kubernetes) :
kubectl run psql-test --image=postgres:14 --restart=Never \
  --env='PGPASSWORD=AppPass123!' -- \
  psql -h $DB_IP -U appuser -d appdb -c '\l'

# Supprimer la stack (supprime TOUTES les ressources)
openstack stack delete hybrid-infra --yes
```

CHAPITRE 8 : GESTION DU CYCLE DE VIE ET RESSOURCES

8.1 Gestion des Quotas par Projet

```
source /etc/kolla/admin-openrc.sh

# Voir les quotas d'un projet
openstack quota show k8s-project

# Modifier les quotas
openstack quota set \
  --cores 64 \
  --ram 131072 \
  --instances 30 \
  --volumes 50 \
  --gigabytes 1000 \
  --floating-ips 20 \
  k8s-project

# Voir l'utilisation actuelle (usage vs quota)
openstack limits show --absolute | grep -E 'Cores|Ram|Instances|Volumes'
```

8.2 Nettoyage des Ressources Orphelines

```
# Volumes non attachés (gaspillage de stockage)
openstack volume list --status available

# IPs flottantes non associées (gaspillage de quota réseau)
openstack floating ip list --status DOWN

# Instances arrêtées depuis longtemps
openstack server list --status SHUTOFF

# ============================================================
# SCRIPT DE NETTOYAGE AUTOMATIQUE
# ============================================================
cat > ~/cleanup-resources.sh << 'CLEAN_EOF'
#!/bin/bash
source /etc/kolla/admin-openrc.sh
echo '=== Suppression des volumes disponibles (non attachés) ==='
openstack volume list --status available -f value -c ID | \
  xargs -r -I{} openstack volume delete {}
```

```
echo '=== Libération des IPs flottantes non associées ==='
openstack floating ip list --status DOWN -f value -c ID | \
  xargs -r -I{} openstack floating ip delete {}

echo '=== Résumé des ressources restantes ==='
openstack server list
openstack coe cluster list
echo '=== Nettoyage terminé ==='
CLEAN_EOF
chmod +x ~/cleanup-resources.sh

# Exécuter le nettoyage
./cleanup-resources.sh
```

8.3 Infrastructure as Code — Versionner avec Git

```
# Initialiser un dépôt Git pour les templates
mkdir -p ~/iac-openstack && cd ~/iac-openstack
git init
git config user.email 'admin@usj.cm'
git config user.name 'USJ OpenStack Admin'

# Copier les templates et la configuration (sans les mots de passe !)
cp ~/heat-templates/*.yaml ~/iac-openstack/

# Créer un .gitignore pour ne pas versionner les secrets
cat > .gitignore << 'EOF'
*openrc*
*passwords*
*.key
*.pem
EOF

git add .
git commit -m 'Initial : template hybrid-stack.yaml + cluster-template'

# Voir l'historique
git log --oneline
```

CHAPITRE 9 : GUIDE DE DÉPANNAGE — TOUTES ERREURS RENCONTRÉES

Erreur 1 : The q-agt/neutron-agt service must be disabled with OVN

CAUSE : Conflit dans local.conf/globals.yml entre l'agent Neutron ML2/OVS (q-agt) et OVN. Ces deux backends réseau sont incompatibles.

SOLUTION Kolla-Ansible : Dans /etc/kolla/globals.yml, s'assurer que :

neutron_plugin_agent: 'ovn'

(Ne jamais activer q-agt avec OVN — Kolla gère cela automatiquement)

```
# Vérifier la configuration OVN dans Kolla
grep neutron_plugin_agent /etc/kolla/globals.yml
# Doit retourner : neutron_plugin_agent: 'ovn'

# Vérifier que les agents OVN sont actifs
openstack network agent list | grep ovn

# Si nécessaire, redéployer uniquement Neutron
source /opt/kolla-venv/bin/activate
kolla-ansible -i ~/all-in-one deploy --tags neutron
```

Erreur 2 : Blocage de init_placement pendant 4h

CAUSE : Mémoire insuffisante. Sans swap, le kernel tue silencieusement les processus (OOM Killer). init_placement et init_neutron tournent en parallèle et consomment plus de RAM que disponible sur une VM 8 Go sans swap.

SOLUTION : Swap de 8 Go créé en Chapitre 3 (ÉTAPE 2) — résout définitivement le problème. Kolla-Ansible est moins gourmand que DevStack grâce à l'isolation Docker.

```
# Vérifier que le swap est actif
free -h
# Échange doit afficher 8.0Gi

# Si le swap n'est pas actif, le réactiver
sudo swapon /swapfile
free -h

# Surveiller la mémoire pendant le déploiement
watch -n 10 'free -h'
```

Erreur 3 : Blocage de l'installation DevStack — résolu par migration vers Kolla

CAUSE ROOT : DevStack sur Ubuntu 24.04 génère des incompatibilités Python 3.12 et des conflits de services difficiles à résoudre sans expertise avancée.
SOLUTION DÉFINITIVE : Migration vers Kolla-Ansible (ce guide). Kolla isole chaque service OpenStack dans son propre conteneur Docker, éliminant tous les conflits.

9.1 Tableau de Dépannage Général

Symptôme	Commande de diagnostic	Solution
Cluster en CREATE_FAILED	openstack stack event list <STACK> --nested-depth 5	Voir l'événement FAILED pour le détail
Node Kubernetes NotReady	kubectl describe node <NODE>	Vérifier les Conditions (DiskPressure, MemoryPressure)
Pod en CrashLoopBackOff	kubectl logs <POD> --previous	Lire les logs du crash précédent
Service Kolla hors ligne	docker ps -a | grep <service>	docker restart kolla_<service>
Erreur d'authentification	openstack token issue	Vérifier source /etc/kolla/admin-openrc.sh
Quota dépassé	openstack quota show k8s-project	openstack quota set --cores 64 k8s-project
Image Glance non trouvée	openstack image list | grep fedora	Re-uploader l'image (voir Étape 14)
Stack Heat ROLLBACK	openstack stack show <STACK>	openstack stack event list --nested-depth 5
Kolla deploy failed=N	journalctl -u docker --since '1h ago'	Vérifier logs Docker et espace disque
API non accessible	curl http://IP:5000/v3	Vérifier docker ps | grep keystone

9.2 Commandes de Diagnostic Avancées

```
# ============================================================
# DIAGNOSTIC KOLLA-ANSIBLE / DOCKER
# ============================================================

# Voir tous les conteneurs Kolla
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'

# Voir les logs d'un service spécifique (ex: Magnum)
docker logs kolla_magnum_api --tail 100 -f
docker logs kolla_magnum_conductor --tail 100 -f
docker logs kolla_heat_engine --tail 100 -f
docker logs kolla_nova_compute --tail 100 -f
```

```
docker logs kolla_neutron_server --tail 100 -f

# Entrer dans un conteneur pour déboguer
docker exec -it kolla_magnum_api bash

# ============================================================
# DIAGNOSTIC OPENSTACK
# ============================================================

# Voir les hyperviseurs et ressources disponibles
openstack hypervisor list
openstack hypervisor show <HYPERVISEUR>

# Voir la console d'une VM (logs cloud-init)
openstack console log show <VM_ID>

# Vérifier les événements d'une stack Heat en profondeur
openstack stack event list <STACK_NAME> --nested-depth 5

# ============================================================
# DIAGNOSTIC KUBERNETES
# ============================================================

# Voir les logs kubelet sur un nœud
ssh -i ~/.ssh/openstack_k8s_key core@<NODE_IP> \
  'sudo journalctl -u kubelet --since "10 min ago"'

# Vérifier les certificats K8s
ssh -i ~/.ssh/openstack_k8s_key core@<MASTER_IP> \
  'sudo kubeadm certs check-expiration'

# Voir les ressources d'un namespace
kubectl get all -n kube-system
kubectl top nodes
kubectl top pods -A
```

9.3 Procédure de Réinstallation Propre

```
# Si le déploiement Kolla a partiellement échoué
source /opt/kolla-venv/bin/activate

# Option 1 : Redéployer uniquement le service en erreur
kolla-ansible -i ~/all-in-one deploy --tags <service>
# Exemples : --tags magnum, --tags heat, --tags neutron
```

```
# Option 2 : Désinstaller complètement et recommencer
kolla-ansible -i ~/all-in-one destroy --yes-i-really-really-mean-it

# Nettoyer Docker
docker system prune -af --volumes

# Relancer le déploiement complet
kolla-ansible -i ~/all-in-one bootstrap-servers
kolla-ansible -i ~/all-in-one prechecks
kolla-ansible -i ~/all-in-one deploy
```

CHAPITRE 10 : SÉCURITÉ ET BONNES PRATIQUES

10.1 Sécurisation des Accès

```
# Créer des rôles différenciés
openstack role create k8s-developer
openstack role create k8s-viewer

# Attribuer les rôles
openstack role add --project k8s-project --user dev-user k8s-developer

# Groupe de sécurité strict pour les nœuds K8s
openstack security group create k8s-strict \
  --description 'Règles restrictives pour nœuds K8s'

# API Server K8s uniquement depuis l'admin
openstack security group rule create k8s-strict \
  --protocol tcp --dst-port 6443 \
  --remote-ip 192.168.100.0/24

# Kubelet uniquement depuis le réseau interne
openstack security group rule create k8s-strict \
  --protocol tcp --dst-port 10250 \
  --remote-group k8s-strict
```

10.2 Bonnes Pratiques DevOps

Pratique	Description	Commande / Action
Versionner les templates	Tous les templates HOT dans Git	git add . && git commit -m 'description'
Valider avant déploiement	Toujours vérifier la syntaxe	openstack orchestration template validate -t file.yaml
Activer le rollback	Protéger contre les déploiements partiels	--rollback-on-failure
Surveiller les quotas	Alerte si > 80% utilisé	openstack limits show --absolute
Sauvegarder les kubeconfigs	Stocker dans un emplacement sécurisé	cp ~/.kube/config ~/backup/kubeconfig-$(date +%Y%m%d)
Documenter les stacks	Utiliser le champ description	Ajouter description: dans chaque template HOT
Rotation des mots de passe	Changer régulièrement	kolla-ansible -i ~/all-in-one reconfigure

CHAPITRE 11 : RÉFÉRENCE RAPIDE — TOUTES LES COMMANDES

11.1 Commandes Kolla-Ansible

```
source /opt/kolla-venv/bin/activate

kolla-ansible -i ~/all-in-one bootstrap-servers  # Préparer les nœuds
kolla-ansible -i ~/all-in-one prechecks           # Vérifier les prérequis
kolla-ansible -i ~/all-in-one deploy              # Déployer OpenStack
kolla-ansible -i ~/all-in-one post-deploy         # Post-configuration
kolla-ansible -i ~/all-in-one reconfigure         # Reconfigurer un service
kolla-ansible -i ~/all-in-one upgrade             # Mettre à jour OpenStack
kolla-ansible -i ~/all-in-one destroy             # Désinstaller TOUT
kolla-ansible -i ~/all-in-one deploy --tags magnum # Redéployer seulement Magnum
```

11.2 Commandes Magnum (Clusters)

```
# Cluster Templates
openstack coe cluster template list
openstack coe cluster template create <NOM> --coe kubernetes --image ...
openstack coe cluster template show <NOM>
openstack coe cluster template delete <NOM>

# Clusters
openstack coe cluster list
openstack coe cluster create <NOM> --cluster-template <T> --node-count N
openstack coe cluster show <NOM>
openstack coe cluster resize <NOM> --node-count N
openstack coe cluster config <NOM> --dir ~/.kube --force
openstack coe cluster delete <NOM>

# Services Magnum
openstack coe service list
```

11.3 Commandes Heat (Stacks)

```
# Validation
openstack orchestration template validate -t fichier.yaml

# Gestion des stacks
openstack stack create <NOM> -t fichier.yaml --parameter cle=val --rollback-on-
failure
```

```
openstack stack list
openstack stack show <NOM>
openstack stack update <NOM> -t fichier.yaml
openstack stack delete <NOM> --yes

# Surveillance
openstack stack event list <NOM> --follow
openstack stack event list <NOM> --nested-depth 5
openstack stack resource list <NOM>

# Sorties
openstack stack output list <NOM>
openstack stack output show <NOM> <OUTPUT_KEY>
```

11.4 Commandes Kubernetes (kubectl)

```
# Nœuds
kubectl get nodes
kubectl describe node <NOM>
kubectl top nodes

# Pods
kubectl get pods -A
kubectl describe pod <POD> -n <NS>
kubectl logs <POD> --previous
kubectl top pods -A

# Déploiements
kubectl create deployment <NOM> --image=<IMAGE> --replicas=N
kubectl rollout status deployment/<NOM>
kubectl scale deployment/<NOM> --replicas=N
kubectl delete deployment <NOM>

# Services
kubectl expose deployment <NOM> --port=80 --type=LoadBalancer
kubectl get svc -A

# Contextes kubeconfig
kubectl config get-contexts
kubectl config use-context <CONTEXT>
```

CONCLUSION ET RÉCAPITULATIF DES PHASES

Phase	Chapitre	Action clé	Résultat
0	3	Swap 8 Go + Mode console	RAM suffisante, fin des blocages
1	3	Mise à jour Ubuntu + Docker	Environnement prêt pour Kolla
2	4	Installation Kolla-Ansible	Outil de déploiement OpenStack prêt
3	4	Configuration globals.yml	Services à déployer définis (OVN, Magnum, Heat)
4	4	Volume Cinder (LVM 30 Go)	Stockage bloc pour les volumes K8s
5	4	kolla-ansible deploy	OpenStack complet déployé dans Docker
6	5	Réseau externe + image Fedora CoreOS	Infrastructure de base prête
7	6	ClusterTemplate + Cluster K8s	Cluster Kubernetes opérationnel
8	6	kubectl get nodes	Accès au cluster, nœuds Ready
9	7	Template HOT hybrid-stack.yaml	Stack VM+K8s déployée en une commande
10	8-10	Gestion, sécurité, monitoring	Infrastructure en production

Checklist de validation finale

☐ free -h : Swap de 8 Go actif
☐ docker ps | grep kolla : tous les conteneurs en état 'Up'
☐ openstack service list : tous les services visibles
☐ openstack coe service list : magnum-api et magnum-conductor 'up'
☐ openstack coe cluster list : cluster en statut CREATE_COMPLETE
☐ kubectl get nodes : tous les nœuds en statut 'Ready'
☐ openstack stack list : hybrid-infra en statut CREATE_COMPLETE
☐ openstack coe cluster resize : scaling fonctionnel

ESSONO EFFA Etienne Florian & NGNINTEDEM FEUPI Vaillant
Sous la supervision de M. NGUIMBUS Emmanuel — Université Saint Jean — 2025–2026